TECHNICAL REPORT

TransportFlow

Real-time public transport for riders and the people who run it

An account of the product, its architecture, its data and security model, and what was verified — written to be read end to end by a technical evaluator who has never seen the repository.

Field	Value
Product version	1.5.0 — Android production build, verified on physical hardware
Report date	31 July 2026
Author	Azer Debbabi
Program	IEEE Computer Society · ISIMM Student Branch
Category	Mobile Development
Repository	github.com/azer-dev1/TransportFlow
Backend	Supabase (hosted, eu-west-2) — no application server
Client	Expo SDK 57 · React Native 0.86 · TypeScript · expo-router
Document status	As-built. Describes the system as shipped, not as planned.

How to read it

The document moves from what the product is, to how it is built, to what was verified and where it falls short. Sections 1 and 2 are the orientation layer. Sections 3 to 5 are the system core architecture, data and live updates, and the security model; if you read only three sections, read these. Sections 6 to 8 cover the real network data, the engineering practice behind the build, and the limits.

It is written as an as-built account. Where the shipped system departs from the original plan, it says so and explains why — those deviations are among the most interesting engineering content in the project, not something to paper over.

1. The project in one page

Public transport riders in Tunisia plan their day around timetables they cannot verify. A delay is usually discovered at the stop, after the wait has already been spent. Operators know about the disruption long before the rider does — there is simply no channel between them.

TransportFlow is that channel. It is a single mobile application serving two audiences at once. Riders browse lines, stops and departures across five real Tunisian networks, see live status on the trip they care about, star favorites and receive push notifications when something changes. Authorized transport staff use the same app to report a delay, a cancellation or a service alert — and that report reaches every affected rider within about a second.

The engineering decision that shapes everything else is that there is no application server. The mobile app talks directly to a Postgres database, and the database itself decides what each request is allowed to read or write. Authorization is not middleware that can be bypassed; it is a property of the data.

1.1 At a glance

Dimension	Value
Application	Android production build 1.5.0, verified on a physical device against the hosted backend
Application servers	Zero — nothing to host beyond Supabase
Server-side code	2 Edge Functions (notify-trip-update, delete-account)
Database	10 SQL migrations · 11 tables + 1 view · 7 callable or scheduled functions
Scheduled jobs	3 pg_cron jobs — trip materialization, trip completion, notification retention
Transit catalogue	78 lines · 474 stops across 5 real Tunisian networks
Live schedule	~2,000 dated trips generated per week from 318 service patterns
Data source	catalogue-data.transport.tn — Tunisian Ministry of Transport open data
Delivery	EAS Build (APK) + EAS Update (over-the-air JavaScript updates)
Git history	Feature branches and reviewed pull requests onto a protected master
Hosting cost	$0 — free tiers throughout

Table 1 — TransportFlow at a glance.

1.2 What makes it different

Four decisions separate this from a conventional CRUD mobile app.

- **The middle tier was deleted.** Authorization lives in Postgres, not in application code. The app carries only a public key plus the signed-in user's token; the database is the gatekeeper. The rules therefore protect every path to the data — the app, a raw REST call, or someone with curl.
- **Staff never edit a trip; they append to a log.** A report is an insert into an append-only table, and a trigger synchronizes the trip's current state in the same transaction. Every state the network was ever in is attributable to a person and a timestamp. Corrections are new rows, never rewrites.

- **One write produces four effects.** That single insert synchronizes trip state, streams a live update to open screens, writes a durable notification for every affected rider, and sends the push — driven by a database webhook rather than by a server.
- **The timetable generates itself inside the database.** Recurring service patterns are expanded into concrete dated trips by a nightly job, with no external scheduler and correct behavior across daylight-saving changes.

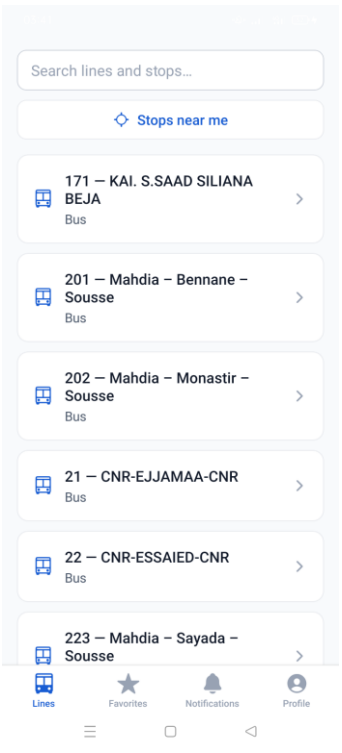
2. What the application does

2.1 For riders

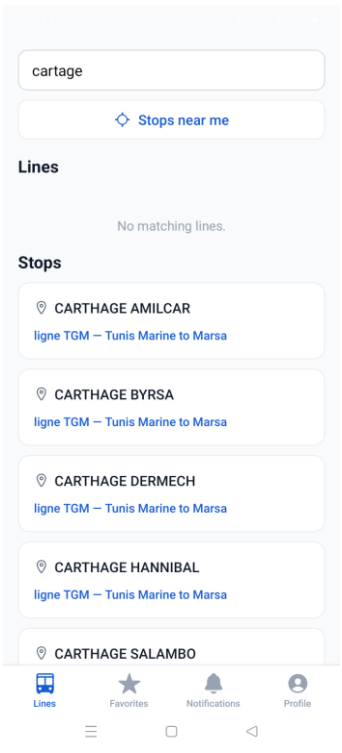
- **Browse and search.** A line list with typo-tolerant search across lines and stops — typing "cartage" finds Carthage — with stop results decorated by the lines that serve them.
- **Line detail.** Route map, ordered stop timeline, active service alerts and upcoming departures.
- **Live trip detail.** Per-stop arrival estimates computed by the database rather than the phone, a status badge, and the full report history. Updates stream in without a refresh.
- **Nearby stops.** Foreground location and a 3 km radius query, shown on a map and as a list with distances and serving-line chips.
- **Favorites.** Star any line or stop; a per-favorite mute bell keeps it starred but excludes it from notifications.
- **Notifications.** A durable in-app list with a live unread badge, plus push notifications that deep-link straight to the affected trip or line.
- **Quiet hours.** A per-user window in local wall-clock time during which pushes are suppressed but the in-app record is still written.
- **Account.** Email and password sign-in, password reset by deep link, and in-app account deletion.

2.2 For transport staff

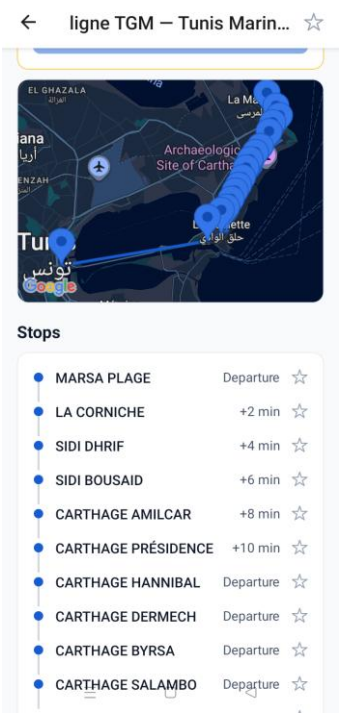
- **Report trip status.** On-time, delayed with a duration, or cancelled — filed against a specific dated trip and permanently attributed to the reporter.
- **Post service alerts.** Line-level announcements that surface as banners on the affected line and fan out to subscribers exactly like a trip report.
- **One app, not two.** Staff surfaces render inline in the rider interface, gated by role in the UI and — authoritatively — by the database.



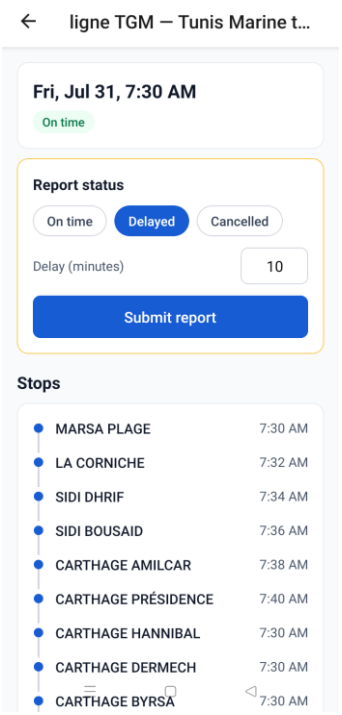
Home — lines across five networks



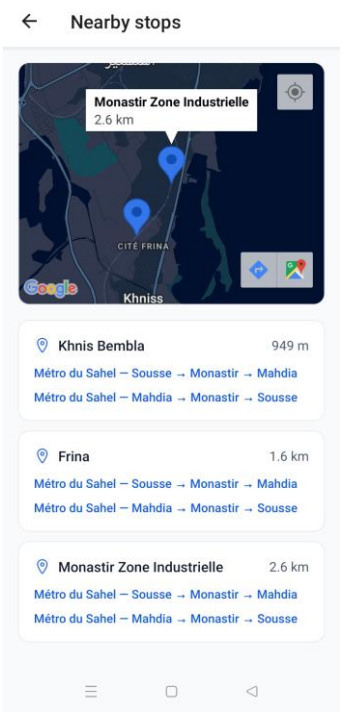
Typo-tolerant search



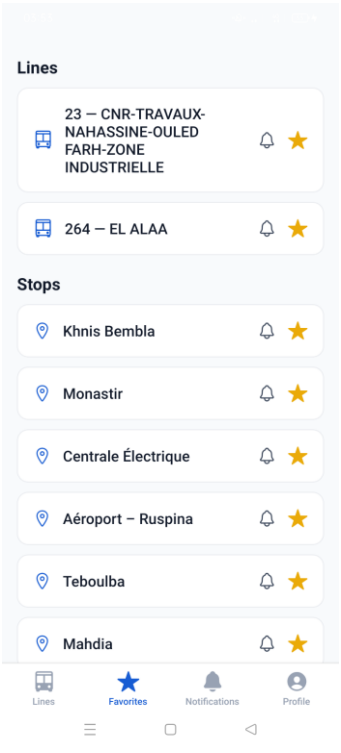
Line detail — map and stop timeline



Live trip — ETAs and staff report card



Nearby stops within 3 km



Favorites with per-item mute

Figure 1 — Captured on-device from the 1.5.0 production build, against the live Tunisian network. The amber-bordered card on the trip screen is the staff-only surface.

3. How it works

Four moving parts: a phone, a Postgres database that is also the authorization layer, two small serverless functions, and a push service.

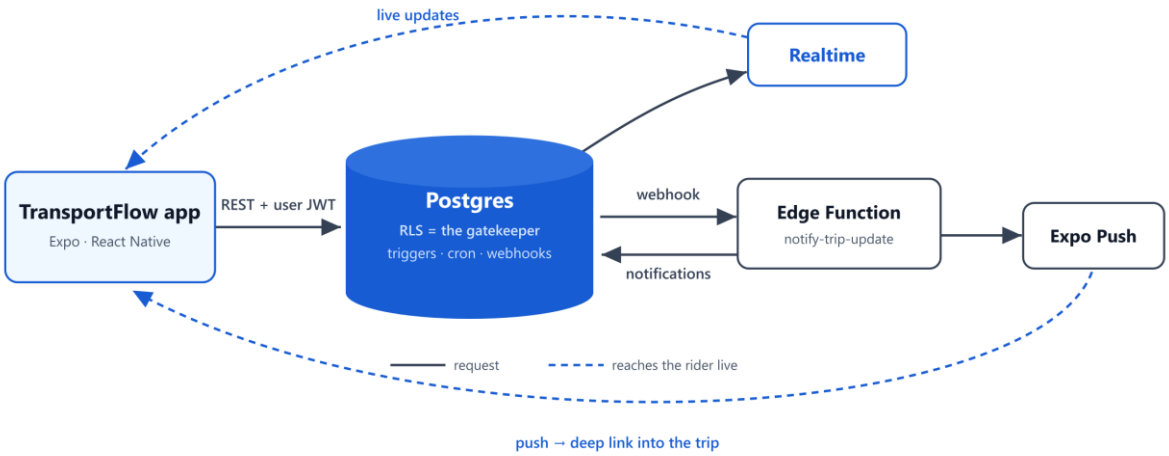


Figure 2 — System architecture. The app carries a public key plus the user's token; Postgres itself is the gatekeeper.

The mobile application is the only client. It authenticates against Supabase Auth, reads and writes over REST, and subscribes to a live channel for changes. Postgres is not merely storage — it holds the authorization rules, the state-synchronization logic, the scheduling, and the outbound webhook that starts the notification fan-out. The two Edge Functions run only when invoked: one by the database, one by the user.

3.1 The central trade-off

Removing the middle tier is the decision everything else follows from, so it deserves an explicit accounting rather than an assertion. Every concern a server would have handled still has an owner.

Concern a server would have handled	How it is covered without one
Authentication	Supabase Auth issues a token; the client attaches it to every request
Authorization	Row-level security policies plus table and column grants, evaluated by Postgres on every statement
Staff-only writes	A policy checking the caller's role, with the reporter identity forced to the authenticated user
Business logic in the write path	Database triggers — state synchronization, duplicate rejection, webhook dispatch
Privileged server-side work	Two Edge Functions holding the service key — never the client
Scheduled jobs	pg_cron, inside the database
Secrets	Supabase Vault; the webhook URL and key never appear in a schema dump or in git

Table 2 — Server responsibilities and their replacements.

What is gained

- One source of truth for authorization — a rule is written once and applies to the app, to a raw REST call, and to anyone poking at the API.
- No API surface to design, build, host, monitor or keep in sync with the schema, and therefore no hosting bill.
- Type safety end to end: types are generated from the live schema, so a schema change that would break a screen fails compilation instead of failing in a user's hands.

What is given up

- There is no place for arbitrary business logic in the request path. Everything must express itself as SQL, a policy, a trigger or a function invoked out of band. Every rule this project needed fitted; a system with heavier request-time computation would feel the constraint.
- Observability is whatever the platform provides — there is no custom logging middleware to instrument.
- The client is coupled to the schema shape. Generated types make this tractable, and over-the-air updates make client fixes fast.

4. Data, live updates and notifications

4.1 An append-only history, not an editable row

The catalogue holds lines, stops and the ordered relationship between them. Service patterns hold the recurring timetable. Trips are concrete dated departures. The interesting table is the one that records what actually happened.

Staff do not update a trip. They insert a row into `trip_updates`, an append-only log, and a trigger synchronizes the trip's cached status in the same transaction. There is no update or delete path on that log, and — the most instructive line in the whole schema — **the trips table has no client-facing write policy at all**. Not a restrictive one, not a staff-only one: none. There is simply no second way to change a trip's status, which is what makes the audit trail unavoidable rather than merely conventional.

The timetable maintains itself. A nightly job expands service patterns — stored as local wall-clock times — into dated trips on a rolling seven-day horizon, and a second job closes out departures that have passed. Storing local time and converting at materialization is what makes the schedule correct across daylight-saving boundaries by construction.

4.2 One insert, four effects

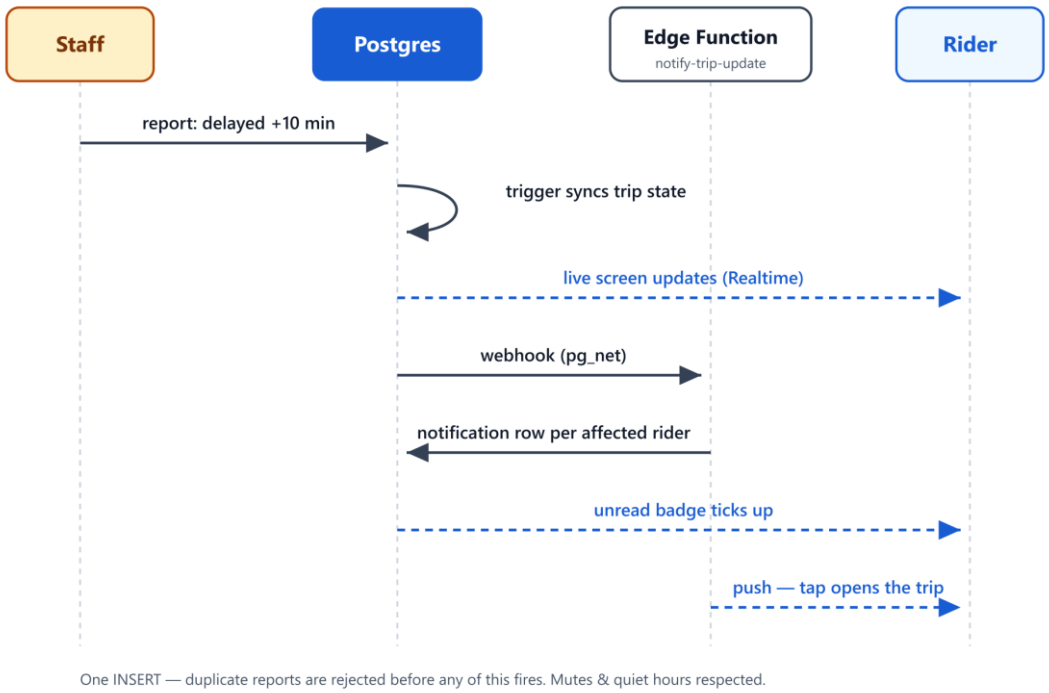


Figure 3 — The notification fan-out. A single staff insert drives state sync, the live stream, durable notification rows and the push.

1. **Staff file a report.** The database requires that the reporter is the authenticated user and holds a staff or admin role. An identical report on the same trip within five minutes is rejected.
2. **The trip is synchronized.** Status, delay, reporter and timestamp are updated inside the same transaction as the log row.
3. **Open screens update live.** Every subscribed client sees the change, and the trip screen refetches — status badge and all per-stop estimates move together, in about a second.
4. **A webhook fires.** A trigger reads the endpoint and key from the secret vault and calls the Edge Function.
5. **Recipients are resolved and recorded.** The function finds every user holding an unmuted favorite on the affected line — or on any stop that line serves — and writes one durable notification row each. Those rows are the record; the unread badge ticks up live, and each rider's live channel carries only their own rows.
6. **Pushes go out, and dead devices are cleaned up.** Batched pushes skip anyone inside their quiet-hours window; tapping one opens the affected trip. Tokens belonging to uninstalled apps are pruned automatically.

Push delivery is allowed to fail. The notification row in the database is the durable record, so a rider who missed the push still finds the update in the app.

5. Security

With no middleware in the request path, security is not "also enforced" in the database — it is only enforced there, which means the layering must be right. It is built in four layers.

- **Grants are the ceiling.** The project does not auto-expose new tables. Every table needed an explicit grant before any policy could open it, so a table added without thought is unreachable rather than accidentally public. The privileged function role was granted exactly the tables it touches and nothing more.

- **Policies narrow within the grants.** A grant says "this role may attempt to read"; the policy says "and only these rows". Catalogue tables are readable with no write policy at all. Personal tables — favorites, devices, preferences, notifications — are restricted to rows owned by the caller.
- **Column grants handle what rows cannot.** Profile updates are revoked wholesale and re-granted on the display-name column alone. A rider attempting to promote themselves to staff receives a clean database permission error — not a silent revert, and not an application-level check that could be bypassed.
- **Privileged paths are few, explicit and audited.** Exactly one client-reachable routine can change a role; it verifies that the caller is an admin and is unavailable to anonymous callers. The functions that must bypass row security are narrow, fixed in behavior and pinned against search-path attacks.

Personal data is minimal by design: an email address, a display name, push tokens and favorites. In-app account deletion cascades the personal rows away while preserving operational history with attribution nulled, so deleting an account never rewrites what the network did.

6. Real Tunisian network data

The app is not populated with invented demo data. Five real networks were imported from the Ministry of Transport open-data portal through a reusable GTFS importer.

Feed	Coverage	Imported
TRANSTU — métro léger and TGM	Tunis	3 lines / 31 stops
SORETRAK — city buses	Kairouan	34 lines / 189 stops
SNTRI — intercity coaches	National	23 lines / 157 stops
SNCFT Métro du Sahel (hand-authored)	Sousse – Monastir – Mahdia	2 lines / 32 stops
STS Sahel buses (converted from 2016 data)	Sousse	16 lines / ~65 stops
Total — 5 feeds	Five networks	78 lines / 474 stops

Table 3 — Imported feeds. From these, 318 service patterns materialize into roughly 2,000 dated trips per week.

Two of the five feeds required engineering work before a single row could be imported, and both are worth stating plainly because data acquisition is usually the least visible part of a transit product.

STS Sousse is published as a .zip that is in fact a RAR archive containing UTF-16 tab-separated files. It is GTFS-shaped but not GTFS: headers are misspelled, dates use a different format, one filename is hyphenated, stop coordinates are absent entirely and stop names are Arabic-only. A dedicated converter repairs each of these and joins coordinates and Latin names from the national stop reference. Only 16 of the feed's 265 routes actually carry trips, and the schedules date from 2016 — both facts are documented rather than hidden.

Métro du Sahel had no digital feed at all: the operator's dataset on the portal has no files uploaded. Rather than drop a network that matters to the Sousse–Monastir–Mahdia corridor, the printed summer 2026 timetable was transcribed into a minimal feed by hand, with the station list taken from the portal's cartography data. The build script records its own provenance, so the hand-authored feed is reproducible rather than a one-off artifact.

The importer's value is not that it parses GTFS. It is that switching cities is now one command.

7. How it was built and verified

7.1 Engineering practice

Practice	Evidence
Feature branches and reviewed pull requests	Every significant feature landed as its own reviewed PR onto a protected master; nothing was pushed straight to the main branch
Conventional commit messages	feat: / fix: / docs: / chore: throughout, producing a history that reads as a changelog
Migrations as the source of truth	10 numbered SQL files, replayable from empty; no schema change exists only in a dashboard
Type safety as a gate	Database types generated from the live schema; a type check across typed routes and typed queries
Verification before merge	Live smoke tests against the running backend at each phase — denials, live loop, constraints, full fan-out
Documentation alongside the code	README, technical documentation, development log, slide deck and speech, plus per-directory READMEs

Table 4 — Engineering practice and its evidence.

The work ran in thirteen phases, and two of them were course corrections worth naming. Expo Go proved unusable on the test device, so a custom development build replaced it. A local Docker backend was consuming the development machine, so the project moved to a hosted one — after which the daily loop no longer needed Docker at all. Both are recorded in the development log rather than smoothed over.

7.2 What was verified

Verification was manual but systematic, and aimed at the riskiest logic rather than the easiest. With no API layer, the highest-risk code is SQL — policies, grants, triggers and functions — so that logic was exercised directly in the database, including its denial paths.

- **Authorization.** A rider was refused when filing a report, when patching a trip, when patching their own role, and when reading another user's notifications — tested by impersonating the role and token claims directly.
- **Append-only guarantees.** No update or delete path exists on the report log; corrections append. Verified by policy inspection and live attempts.
- **The live loop.** A staff report appears on an open rider trip screen within about a second, updating both the badge and every stop estimate — tested across two devices against the hosted backend.
- **The fan-out, three ways.** Normal (recipient found, row written, push sent, stale token pruned), quiet hours (row written, push skipped) and muted (no recipients) — inspected through the function's own responses.
- **Scheduling and time.** Both cron jobs run on schedule and maintain a rolling seven-day horizon; local departure times convert to the correct instants, including cleanup of 22 off-by-one-hour phantom trips found during import.
- **Delivery and lifecycle.** A push arrives on the lock screen and deep-links to the right trip; uninstalled devices are pruned against the real push API; account deletion cascades personal data while preserving attribution-nulled history.

> **What this does not cover: regression.** Every result above is a point-in-time check. Nothing prevents a future migration from silently widening a policy, and nothing would catch it except repeating the checks by hand. That is precisely why an automated test suite heads the roadmap rather than trailing it.

8. Limitations

None of the following are accidents. Each is a scope decision with a stated reason, and each is expanded in the full report.

Limitation	Why	Consequence
"Real-time" means staff-reported, not vehicle GPS	Most Tunisian fleets are not GPS-equipped; a GPS-only design would exclude exactly the networks this product is for	Accuracy depends on operators reporting. The model is format-ready for vehicle feeds where one exists
One direction per line	The importer keeps each route's dominant stop sequence	The return direction is not modelled
No holiday calendar	Calendar exceptions are ignored by the importer	School and holiday closures are not reflected
Uniform delay across stops	A single delay value shifts every downstream stop equally	A delay that recovers partway along a route is over-stated later on
No automated regression tests	Time was spent on verification breadth instead	The highest engineering risk in the project; heads the roadmap
Light theme only, no offline mode	Scope decisions against a fixed timeline	A rider with no connectivity sees whatever was last loaded

Table 5 — Principal limitations.

Two operational risks are worth naming alongside them: free-tier ceilings on live connections and function cold starts are acceptable at demo scale but unmeasured beyond it, and the project has a single maintainer — mitigated by migrations that replay from empty, an idempotent importer and a written runbook.

8.1 What comes next

#	Next step	Why it is next
1	Automated test suite	The single largest gap — a policy suite asserting every denial path, plus mobile journey flows, wired into CI
2	Live vehicle positions (GTFS-RT)	Turns operator-reported status into genuine tracking where a feed exists — one operator already publishes one
3	Both directions per line	The most visible modelling limitation for riders
4	Per-reporter rate limiting	Completes the anti-abuse story beyond the five-minute duplicate guard
5	Play Store release	The natural next distribution step; account deletion is already implemented for compliance

Table 6 — The first five roadmap items, in the order they would actually be built.

Two things are worth noting about that ordering. The top item is not a feature: a project that ships user-visible work faster than it builds confidence in what already exists eventually stops being able to change safely. And the second item is cheap precisely because of decisions already taken — ingestion, storage, fan-out and delivery all exist, so a vehicle feed is a new data source rather than a new pipeline.

9. Where to look next



github.com/azer-dev1/TransportFlow